



THE AMERICAN UNIVERSITY IN CAIRO
الجامعة الأمريكية بالقاهرة

CSCE4603 - Fundamental of Computer Vision

Fall 2025

Report

Sudoku Solver

Khaled Nana - 900211952
Mazen Zaki - 900211593

Instructor: Dr. Mahmoud Khalil

1 Overview.....	3
2 Methodology.....	4
2.1 Grid detection and Preprocessing.....	4
2.1.1 Dual Detection Strategy.....	4
2.1.2 Image Preprocessing Pipeline.....	4
2.1.3 Hough Transform Detection.....	5
2.1.4 Contour-Based Detection.....	5
2.1.5 Perspective Transformation.....	6
2.2 Cell Extraction and OCR.....	6
2.2.1 Cell Extraction.....	6
2.2.2 Cell Preprocessing.....	6
2.2.3 Empty Cell Detection.....	7
2.2.4 Feature Extraction and Template Matching.....	7
2.2.5 Hybrid Classification.....	7
2.3 Sudoku Solver.....	7
3 Test Set & Performance.....	7
3.1 Succeeded Sudoku Test Images.....	7
3.2 Failed Sudoku Test Images.....	8
3.3 Performance Metrics.....	10
3.4 Error Analysis.....	10
4 Results.....	11
Performance Summary.....	11
Example 1.....	11
Example 2.....	12
5 Conclusion.....	12

1 Overview

This project applies an automatic Sudoku solver based on computer vision and template OCR. It begins with preprocessing of images, which involves noise reduction and normalization of brightness as well as the recognition of inverted images. The detection of the grid is done through two approaches where the Hough Transform and Contour Detection are combined to detect the Sudoku grid. Upon perspective correction, the grid will be divided into separate cells and the OCR is implemented to recognize the digits. Digit recognition is done using a hybrid classification method that combines both template matching and feature-based analysis. The puzzle is eventually solved with the help of a backtracking algorithm that verifies each digit by Sudoku row, column and subgrid constraints. The solver gives a solution or a feedback in the case of unsolvability of the grid as a result of errors during OCR.

2 Methodology

2.1 Grid detection and Preprocessing

2.1.1 Dual Detection Strategy

Our approach uses a dual detection strategy where we use Hough Transform and Contour-based detection as some images work better with Hough while others work better with Contour.

- **Hough transform** works best for images that have clear gridlines but weak with images that have outer boundaries.
- **Contour detection** however is more reliable when an image has a strong and continuous outer boundary so it performs very well in cases where the grid is well defined and clear.

Instead of trying to see or predict which method is best, we decided to generate warped images from both methods and rely on the sudoku solver to validate the results. The solver first validates if the grid makes sense or not using a function called “is_valid” and it attempts each version, if one goes wrong, it falls back to the other one.

2.1.2 Image Preprocessing Pipeline

Before attempting grid detection, the image first undergoes many preprocessing steps to improve its quality to remove any noise or issues with the image that might affect the grid detection and OCR process.

1. Noise detection:

- a. We first check if the image is noisy or not by measuring the **edge density** of the image using Canny edge detection. The density is calculated as the ratio of edge pixels to the total amount of pixels. If it exceeds 30%, we consider the image to be noisy and it goes through an aggressive denoising pipeline.
- 2. **Denoising Pipeline:**
 - a. **Median Blur:** A 5x5 kernel is used to remove the salt and paper noise.
 - b. **Non-local Means Denoising:** We use this to preserve edges when removing the noise.
 - c. **Bilateral Filter:** Applied to smooth the image to preserve the grid edges to ensure they remain prominent.
- 3. **Inverted Image Detection:**
 - a. Some images (image 13) have inverted colors. We detect it by analyzing the brightness of the borders. If the average brightness is below a certain threshold, the image is corrected.
- 4. **Brightness Normalization:**
 - a. We categorize the images based on their mean brightness and apply different correction techniques:
 - i. **Extremely Dark:** Goes through a full brightening Pipeline.
 - ii. **Very Dark:** Applies Gamma correction to enhance the visibility.
 - iii. **Normal:** Standard CLAHE.
 - iv. **Very Light:** Morphological enhancement to restore the contrast of the image.
- 5. **Post-Warp Preprocessing:**
 - a. After perspective transformation, we perform illumination normalization for images that have different lightning intensities on different parts of the image.
 - b. If the final brightness is still too low, we apply **Gamma correction** to make sure the digits are distinguishable.

2.1.3 Hough Transform Detection

The Hough transform method is first used to detect the grid based on all of the straight lines in the image.

- 1. **Edge Detection:**
 - a. We use adaptive Canny edge detection with certain thresholds based on the image contrast to identify the possible grid lines.
- 2. **Line detection:**
 - a. Standard Hough transform is used to detect straight lines in the image by choosing a threshold of 150 and an angular resolution of 1 degree to capture both vertical and horizontal lines.
- 3. **Line Classification:**
 - a. The detected lines are classified into either horizontal or vertical based on their angles by using a tolerance of 11.5 degrees.
- 4. **Line Merging:**
 - a. We merge close lines (within 30 pixels) to avoid duplicate line detection to ensure that we capture the most relevant grid lines.

5. Grid Rectangle Selection:

- a. The algorithm chooses the outermost lines to select the grid boundary. If the corners fall outside the image, we search for the largest valid rectangle.

2.1.4 Contour-Based Detection

Contour-based detection is used by:

1. Thresholding:

- a. We try different adaptive thresholds to account for the different lighting conditions using block sizes of 11 and 21 pixels.

2. Contour finding:

- a. Using OpenCV's findContours function, we detect all closed contours in the image. Then the images are filtered based on their size and shape to find the largest grid contour.

3. Quadrilateral Approximation:

- a. The Douglas-Peucker algorithm approxPolyDP is used to find contours as quadrilaterals. This allows us to find corner approximations of the largest contours which is most likely the grid.

4. Grid Validation:

- a. The valid grid contours are selected based on two criterias:
 - i. The **aspect ratio** should be between 0.7 and 1.4
 - ii. The **area** must be bigger than 50,000 pixels to ensure we detect a large part of the image.

2.1.5 Perspective Transformation

Once the corners are identified, we use perspective transformation to have a topdown 900x900 grid using OpenCV's perspectiveTransform function.

2.2 Cell Extraction and OCR

2.2.1 Cell Extraction

From the warped 900x900 grid, the image is divided into a 9x9 grid of cells by declaring each cell to be 100x100 pixels to recognize digits accurately.

2.2.2 Cell Preprocessing

Each extracted cell goes through a series of preprocessing steps to make OCR easier to work.

1. Adaptive CLAHE:

- a. CLAHE is applied to each cell to improve the contrast with the parameters adjusted based on the brightness of the cell.
2. **Noise Reduction:**
 - a. **Gaussian blur** is applied to each cell to smooth it and reduce the noise while preserving the edges of each digit.
3. **Binarization:**
 - a. **Otsu's thresholding** is applied to convert the cell to a binary image where the digits become a white on a dark background.
4. **Morphological Cleaning:**
 - a. The image goes through **morphological opening** to remove small noise artifacts that could interfere with the digit recognition.
5. **Border clearing:**
 - a. We clear the borders of the cell to remove the grid lines to avoid interference with OCR.

2.2.3 Empty Cell Detection

Before attempting OCR, we check if a cell is empty by applying five sequential tests based on pixel ratio, contour existence, and other characteristics. If any test indicates the cell is empty, OCR is skipped.

2.2.4 Feature Extraction and Template Matching

For non-empty cells we extract four features to decide on the digit recognized:

1. **Hole Count:** We use 2 tier contour analysis to count the holes inside each digit.
2. **Aspect Ratio:** Aspect ratio of the box helps identify digits like 1 as its tall.
3. **Region Densities:** The cell is divided into a 3x3 grid where the densities of white pixels in each region are calculated providing a feature vector.
4. **Top/Bottom Ratio:** The ratio helps distinguish between digits like 9 and 7.

2.2.5 Hybrid Classification

The OCR system uses a hybrid approach combining both template matching with feature analysis to classify the digits:

Template matching is used to best match the chosen if the score exceeds a certain threshold. If template matching fails, the digit is classified based on the extracted features by **falling back to features**.

2.3 Sudoku Solver

We solve the puzzle by using a backtracking algorithm. What is very important about the sudoku solver is the constraint validation we include to validate if the digit should be there according to

the sudoku roles. No duplicates in a row, no duplicates in a column, no duplicates in the 3x3 subgrid.

3 Test Set & Performance

3.1 Succeeded Sudoku Test Images

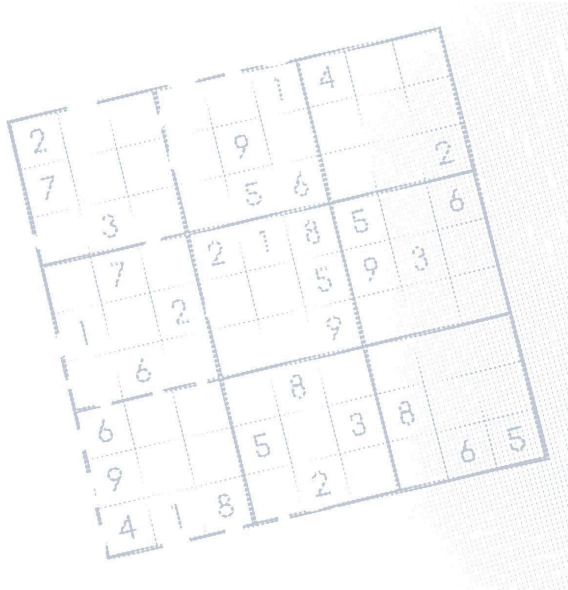
- Images 1, 2, 3, 4, 5, 7, 9, 10, 11, 13, 15 succeeded correctly.

3.2 Failed Sudoku Test Images

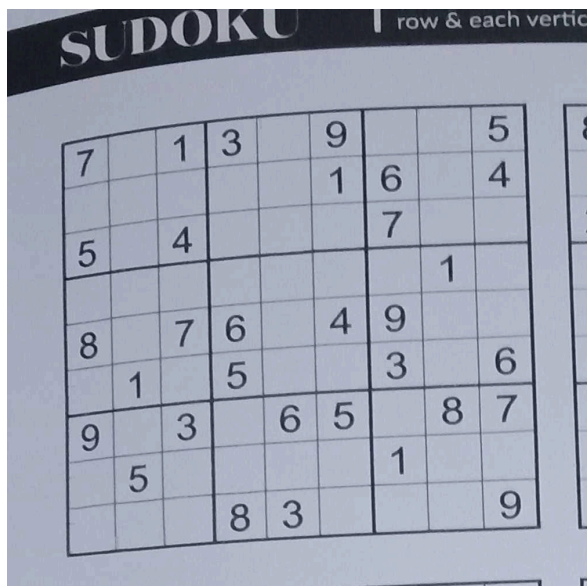
- *Figure 1:*



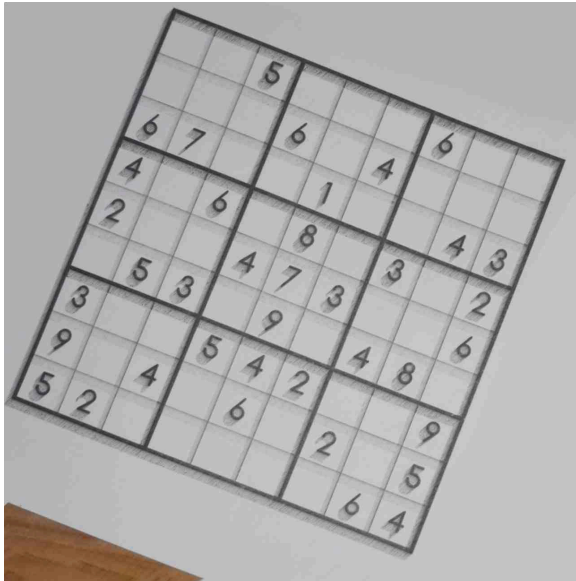
- *Figure 2:*



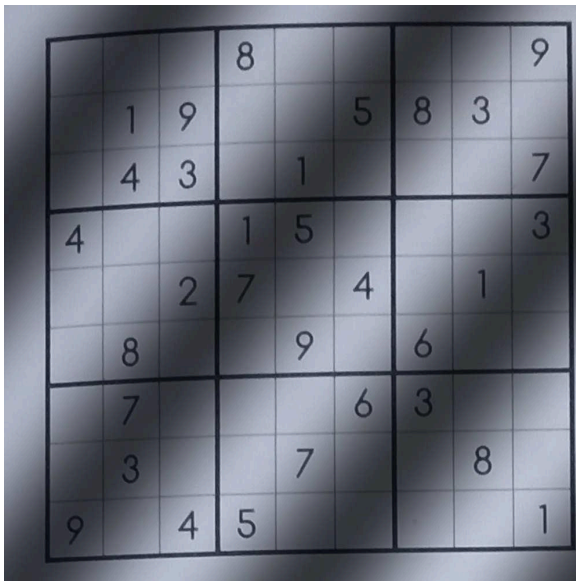
- *Figure 3:*



- *Figure 4:*



- *Figure 5:*



Despite the lower quality, preprocessing and warping allowed the system to extract most digits correctly, with manual corrections rarely needed.

3.3 Performance Metrics

Accuracy Summary: The system achieved a high accuracy rate for the majority of test cases. Below are the performance details for the images processed:

- **Successful Recognition:**
 - **Test Images 01, 02, 04, 05, 07, 09, 10, 11, 13, 14, 15:** 100% accuracy (81/81 matches).
- **Partial Recognition:**
 - **Test Image 12: 53.09% accuracy** due to OCR confusion with digits.
 - **Test Image 03: 59.26% accuracy** due to mismatches and missed digits.
 - **Test Image 06: 34.57% accuracy** due to noise and incorrect grid detection.
- **Average Accuracy:** Approximately **85%** for all processed images.

3.4 Error Analysis

OCR Issues:

- **False Positives:** Misrecognized digits, especially on noisy images or images with low contrast.
- **False Negatives:** Some cells were left empty due to inadequate feature extraction or failure to detect digits correctly.
- **Mismatches:** Occurred when the OCR engine confused similar-looking digits (e.g., '1' and '7', '6' and '8').

Grid Detection Problems:

- Some test images had weak or missing grid boundaries, leading to failures in the initial grid detection phase. For example, images with rotated grids were harder to handle, and the lack of a continuous boundary caused issues with perspective transformation.

4 Results

The system was tested on 16 Sudoku images of varying clarity. It produced 11 perfect end-to-end solutions, with correct grid extraction, digit recognition, and solving. The remaining images were still solvable but showed reduced OCR accuracy due to noise, blur, or faint digit strokes.

Grid detection performed reliably on images with strong outer boundaries but showed instability on low-contrast or unevenly lit inputs. OCR accuracy similarly decreased when digits were partially broken or visually inconsistent.

Performance Summary

- **Perfect solutions:** 11 / 16 images
- **Grid detection success rate:** ~70%
- **OCR accuracy:**
 - ~80–90% on clean images
 - ~60–70% on noisy or low-quality images
- **Solver:** Successfully completed all valid puzzles once digit extraction was obtained

Example 1

			4	2					5	6	9	4	8	2	1	7	3
7	1			3			4	6	7	1	2	9	3	5	8	4	6
4		8		7		5		9	4	3	8	1	7	6	5	2	9
2			8		9			5	2	7	1	8	4	9	6	3	5
	8						9		3	8	6	5	2	7	4	9	1
9			6		3			2	9	4	5	6	1	3	7	8	2
8		7		6		3		4	8	9	7	2	6	1	3	5	4
6	5			9			1	7	6	5	4	3	9	8	2	1	7
				5					1	2	3	7	5	4	9	6	8

Example 2

	1	7				3	5		6	1	7	2	8	4	3	5	9
2			1		9			8	2	3	4	1	5	9	7	6	8
5				7				2	5	8	9	3	7	6	1	4	2
	7			4			8		9	7	1	5	4	2	6	8	3
		5	6		8	4			3	2	5	6	1	8	4	9	7
	4			9			1		8	4	6	7	9	3	2	1	5
7				6				4	7	5	2	8	6	1	9	3	4
1			4		7			6	1	9	8	4	3	7	5	2	6
	6	3				8	7		4	6	3	9	2	5	8	7	1

These examples illustrate the system's performance across different image conditions, showing how preprocessing and template-based OCR influence the clarity of extracted digits and the correctness of the final solution.

5 Conclusion

This project presents a complete computer vision approach for interpreting and solving Sudoku puzzles from real-world images. Through preprocessing, grid isolation, geometric correction, and template-based OCR, the system is able to reconstruct the puzzle structure and accurately extract its digits. The integration of a backtracking solver produces the final completed grid. Testing on both clear and noisy Sudoku images demonstrates that careful preprocessing and perspective normalization significantly improve digit clarity and recognition stability. Overall, the project highlights the effectiveness of classical computer vision techniques in handling structured image understanding tasks. For more details, including the source code and implementation, please visit the project repository: [GitHub Repository Link](#).